



Minhaj
University
Lahore

Mini Compiler for Simple Language

Project Documentation

COMP343 — Compiler Construction

Group Members

1. Maryam Ameen
2. Mahnoor Ali
3. Amina Fiaz
4. Ariba Fatima
5. Aqeela Tahir

1. What is a Mini Compiler?

A compiler is a program that reads code written by a human (called source code) and translates it into a form the computer can actually run. Think of it like a translator between two languages: the language we write in and the language the machine understands.

Our Mini Compiler does this for a simplified programming language. It takes our code step by step through several stages, catches any mistakes (like grammar errors or using a variable before defining it), and finally produces output or executable instructions.

Even though it is called “mini”, it covers all the core stages a real-world compiler goes through which will make it a complete learning project.

How does a Compiler Work in Simple Terms?

Imagine writing a sentence in English and giving it to a translator who:

- Reads each word one by one (Lexical Analysis)
- Checks if the sentence follows the rules of grammar (Syntax Analysis)
- Makes sure the meaning is valid (Semantic Analysis)
- Rewrites it in a simpler form for translation (IR Generation)
- Cuts out unnecessary words (Optimization)
- Delivers the final translation (Code Generation)

Each of these steps is a “phase” of the compiler. If something goes wrong at any phase, an error is reported and the process stops.

2. Workflow Diagram

The workflow diagram below shows the overall path our source code takes from the moment we write it to the moment we see a result. Each box is a stage, and the arrows show what happens next. Error paths branch off when something goes wrong.

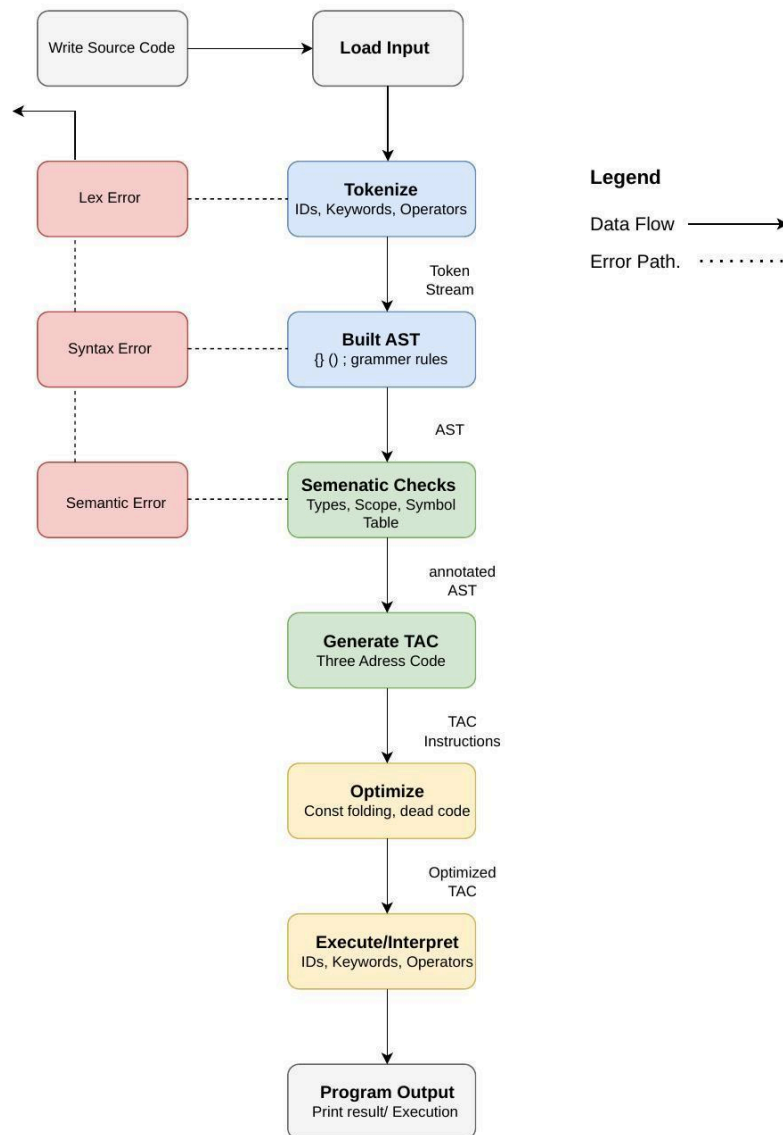


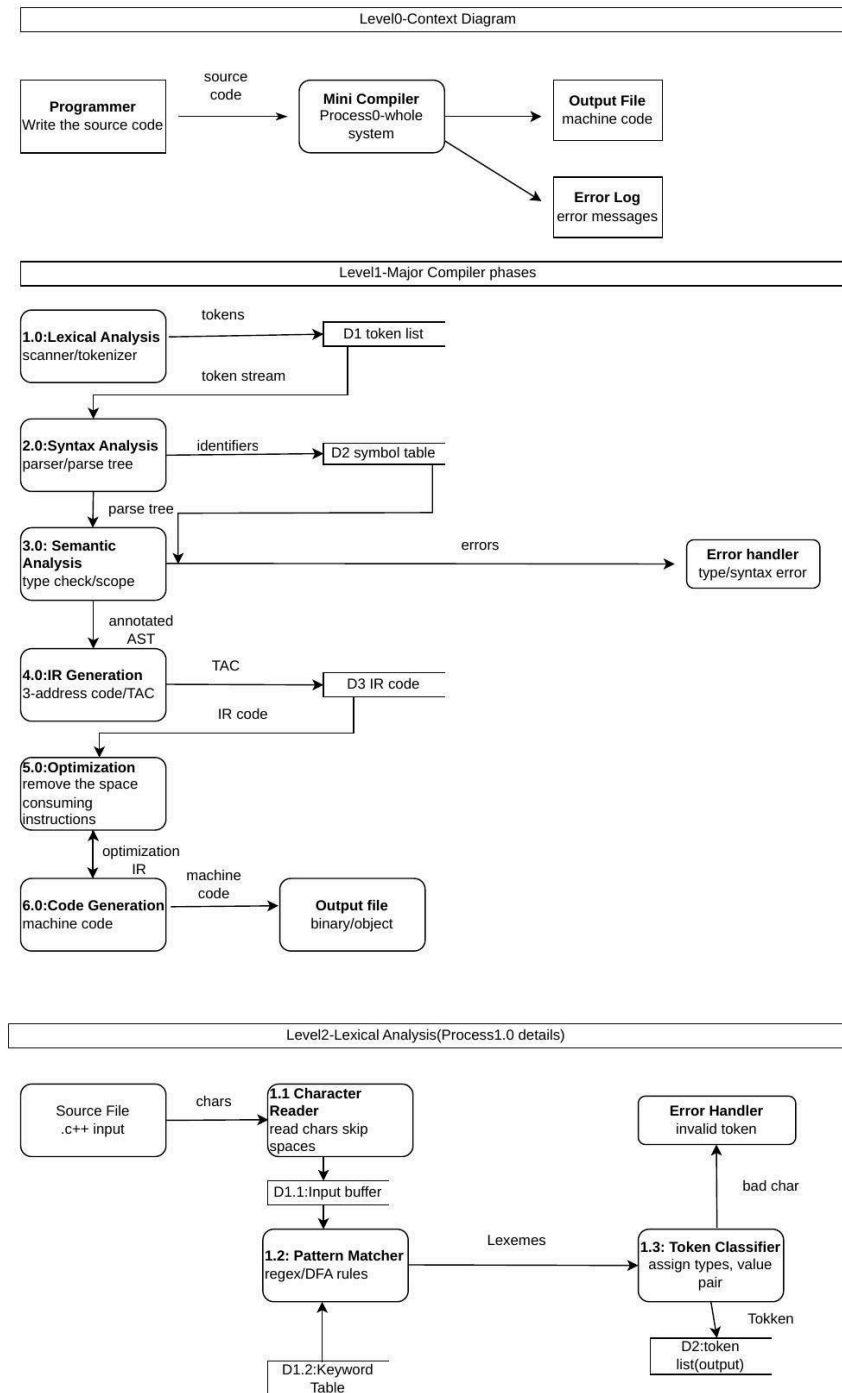
Figure 1: Mini Compiler Workflow Diagram

Explanation:

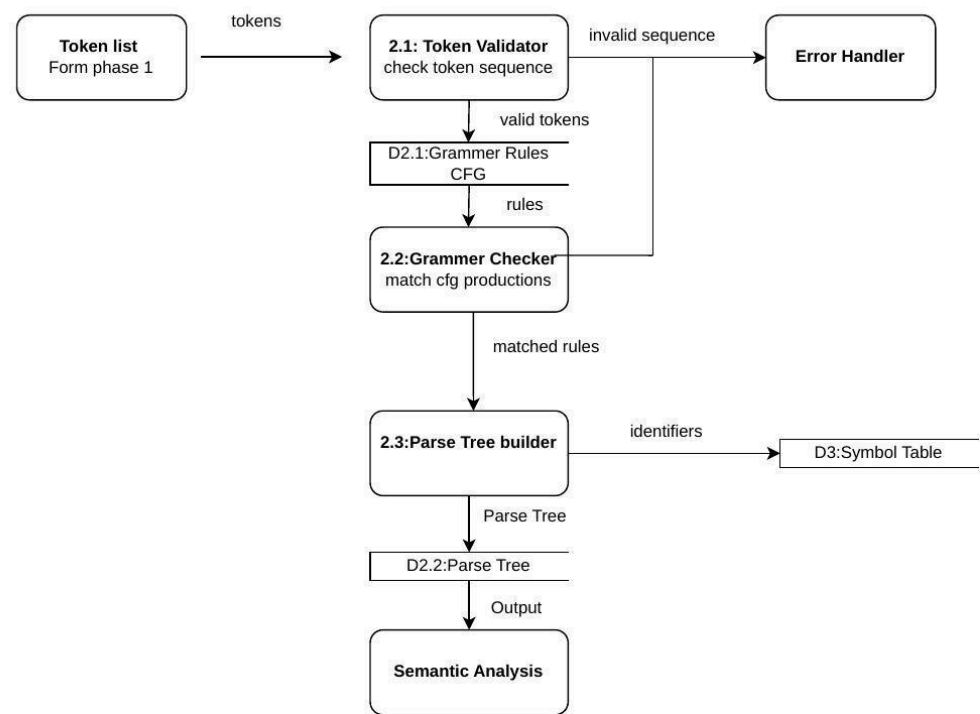
Starting from the top: we write source code, which flows into tokenization (breaking it into words), then into building a syntax tree, then into semantic checks, then generating three-address code (TAC), optimizing it, and finally executing it to produce program output. Any error at any stage redirects to an error message box.

3. Data Flow Diagram (DFD)

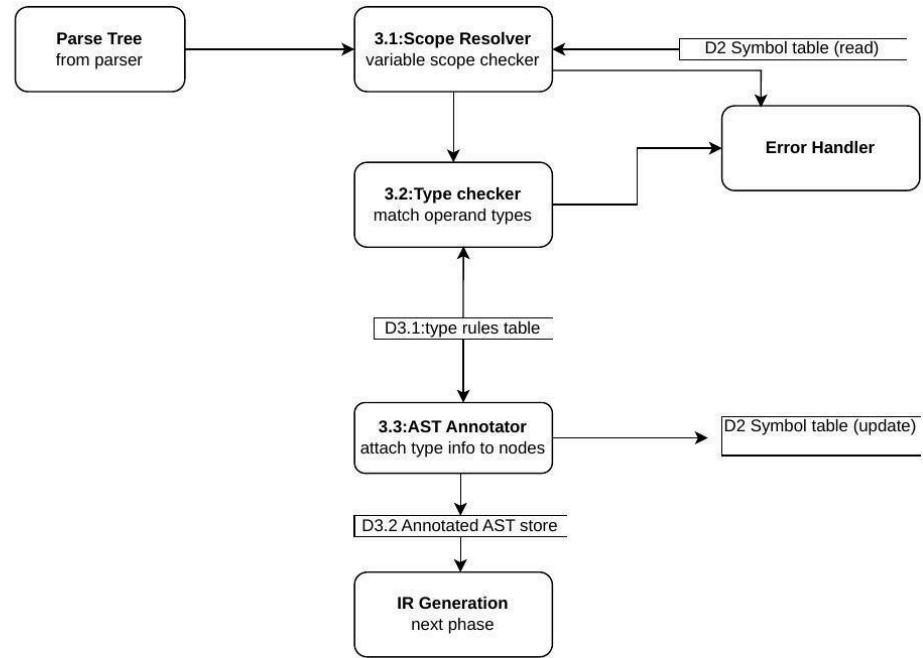
A Data Flow Diagram (DFD) shows how data moves through the system, what comes in, what gets stored temporarily, and what goes out. It is organized in levels: Level 0 gives the big picture, and Level 1/2 zoom into each phase.

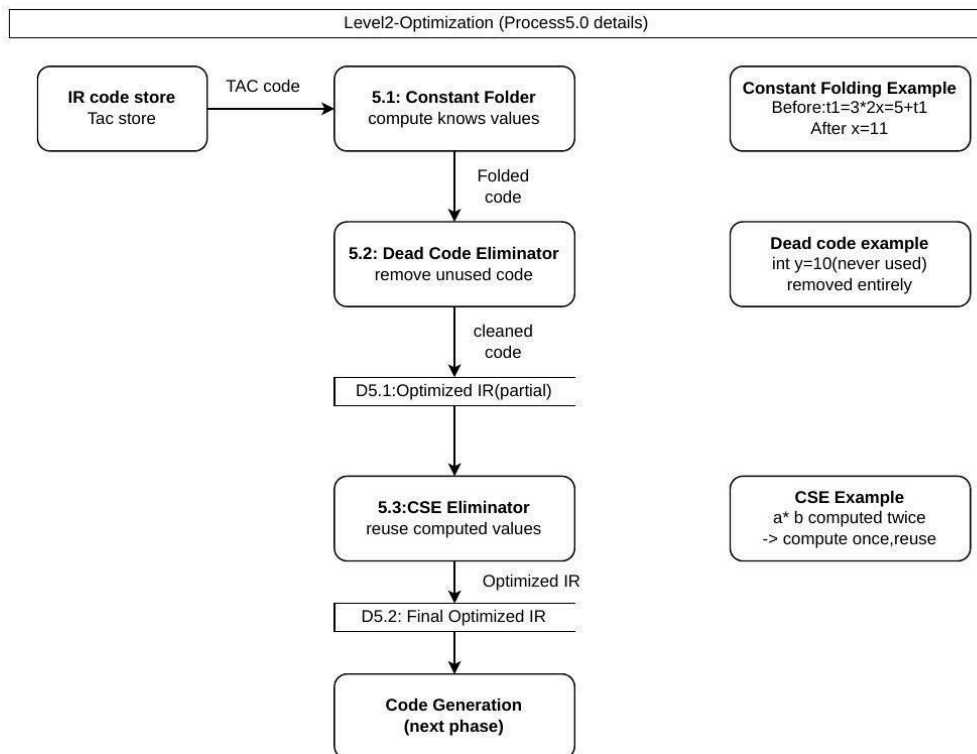
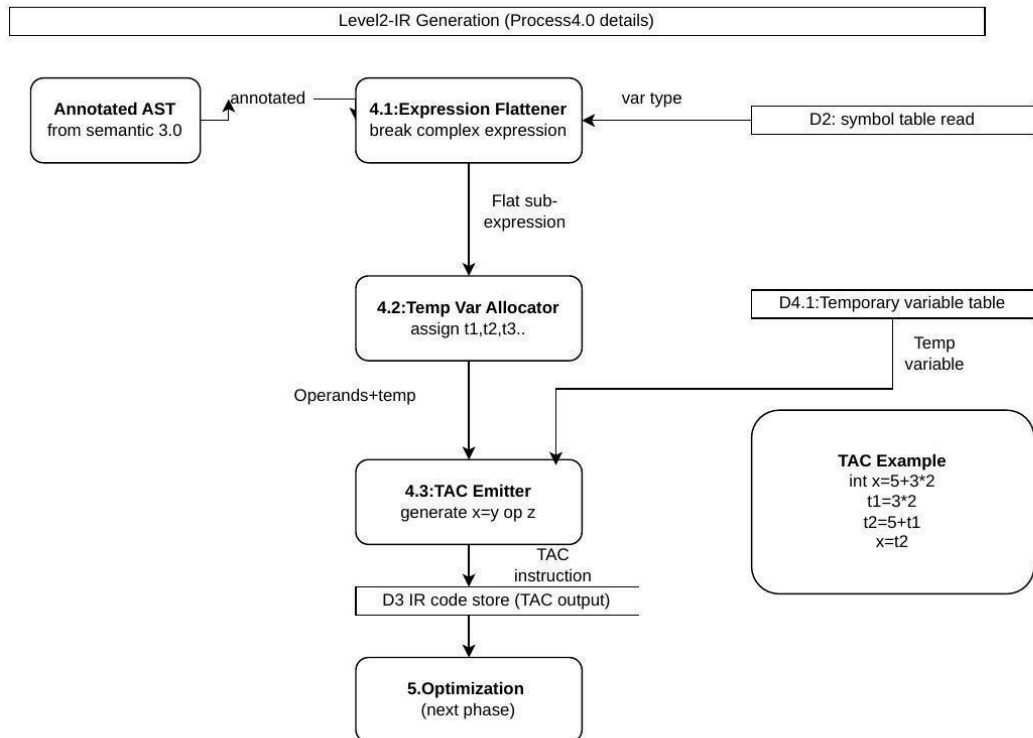


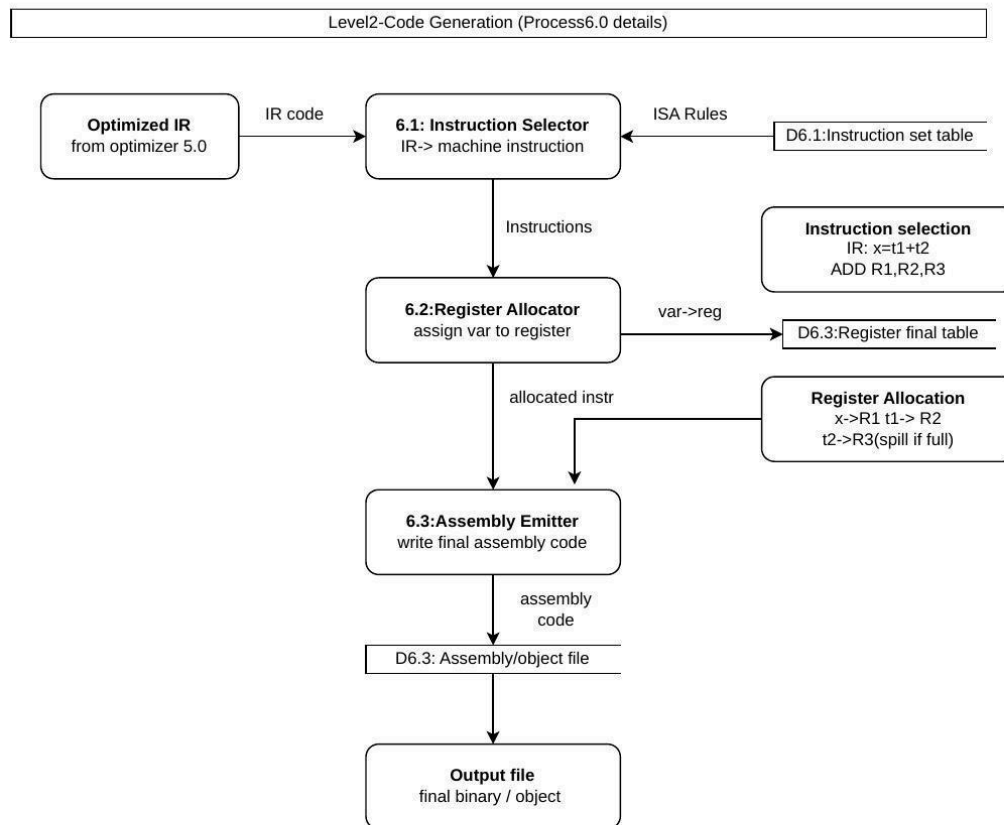
Level2-Syntax Analysis(Process2.0 details)



Level2-Semantic Analysis(Process3.0 details)



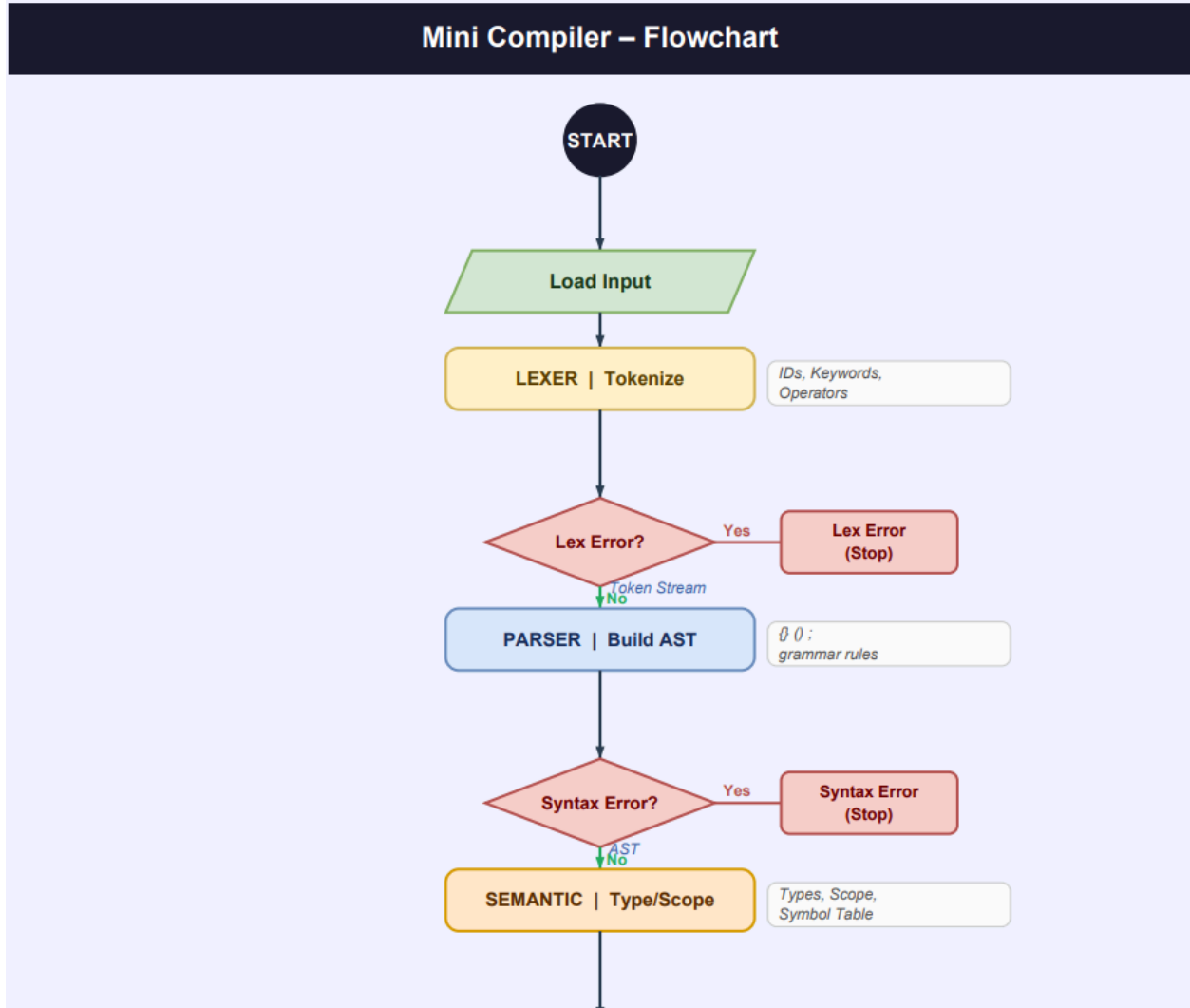


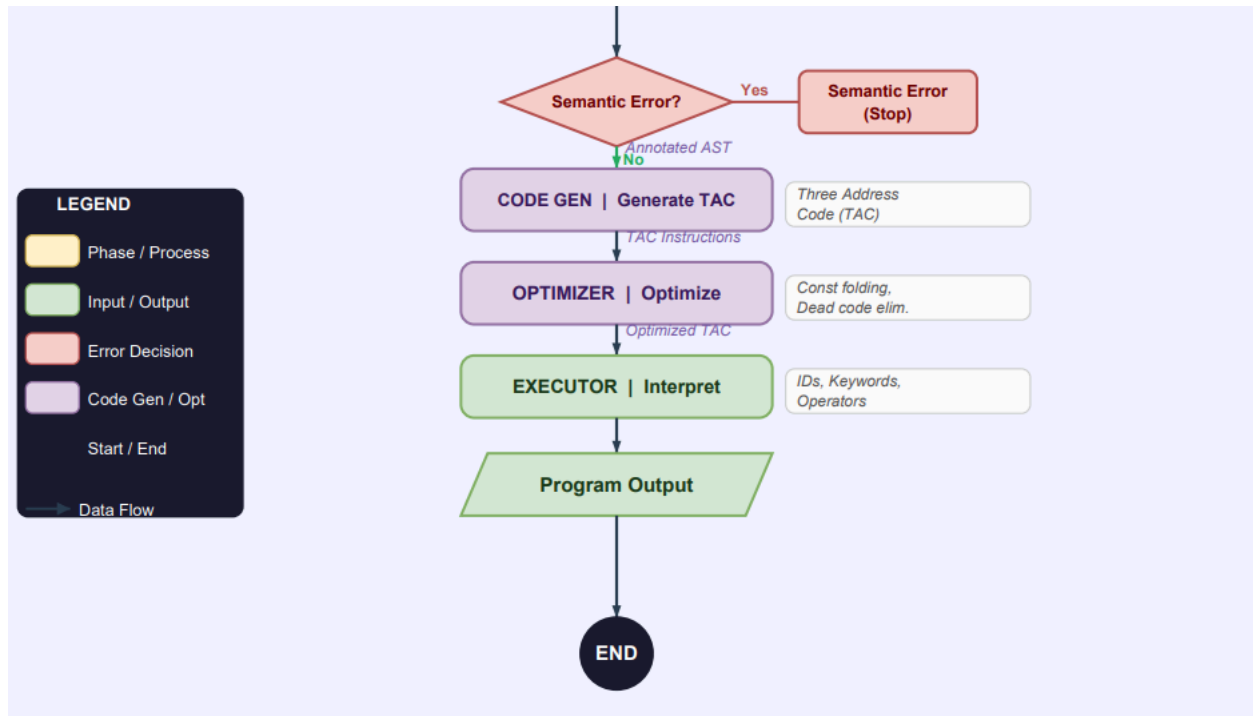


- Level 0 shows the system as a single process: the programmer sends in source code, and the compiler returns either output or error messages.
- Level 1 breaks this into the six major phases, showing how data stores like the token list, symbol table, and IR code connect them.
- Level 2 zooms into individual phases. Lexical analysis reads characters, matches patterns, and classifies them as tokens. Syntax analysis then validates those tokens against grammar rules and builds a parse tree.
- Semantic analysis checks that the meaning of the code is valid — correct types, declared variables, proper scope. IR Generation then converts the annotated tree into flat three-address code (TAC). Optimization simplifies TAC by folding constants, removing dead code, and reusing common sub-expressions.
- Code generation takes the optimized IR and produces the final machine-level instructions by selecting the right CPU instructions, assigning variables to registers, and emitting the assembly output file.

4. Compiler Flow Chart

The flow chart below shows the same process as a decision-based flow, including the yes/no decision points where errors cause the process to exit early. This makes it easy to trace what happens at each step depending on whether the input is valid or not.





Mini Compiler Flow Chart

5. Summary

Here is a quick summary of each phase in plain language:

Phase	What it does	Simple Example
Lexical	Breaks code into tokens	"int x = 5;" → [int][x][=][5][;]
Syntax	Checks grammar structure	Builds a tree showing how tokens relate
Semantic	Validates meaning	x = "hello" + 5 → Type Error
IR Gen	Creates simple instructions	t1 = 3*2, t2 = 5+t1, x = t2
Optimize	Removes wasteful code	x = 11 (constant folded directly)
Code Gen	Produces machine code	ADD R1, R2, R3 (CPU instruction)

This mini compiler covers every stage a real compiler goes through, just on a smaller scale. By working through Lexical Analysis, Syntax Analysis, Semantic Analysis, IR Generation, Optimization, and Code Generation, we get a complete picture of how code is transformed from plain text into something a machine can execute.

The workflow diagram, DFD, and flow chart together show this transformation from three angles: what the steps are, how data flows between them, and how decisions are made at each stage.